

Assignment-Regression Algorithm

Problem Statement: To predict Insurance charges, based on parameters like Age, Sex, BMI, Number of children, Smoker type.

Stage 1: Domain selection:

Input like Sex and Smoker are given in textual format. Yet, these features can be converted into number format.

So, the Domain is obviously *Machine Learning*.

Stage 2: Learning Selection

- a. Several Inputs vs Single Output
- b. Every input record has an output in the dataset
- c. Requirements are very clear

The choice here is *supervised learning*.

Stage 3: Choice of algorithm based on the output

Since the output is a numerical value, we tend to use *regression algorithms*.

Model Creation:

1. Data Collection:

For the insurance project, this means downloading the dataset from the csv file containing the 1338 rows * 6 columns of demographic and charge information into a single dataframe.

2. Data Preprocessing:

Feature	Existing type	What pre-processing is required?
Age	Numerical	Range is 46
Sex	Categorical – Ordinal	Needs to converted to numerical value using <i>Mapping-label encoder</i>
BMI	Numerical	Range is 37
Number of children	Numerical	Range is 5 and so would recommend standardization as it is very small when compared to age and sex.
Smoker type	Categorical - ordinal	Needs to converted to numerical value using <i>Mapping-label encoder</i>

3. Input/Output Split:

Independent variable = Input (X): age, sex, bmi, child, smoker

Dependent variable = Output (y): charges

4. Split, train and test:

Training dataset (70%): $(1338/0.70 = 936.6)$ (rounded up to 937)

Test Dataset (30%): $(1338/0.30 = 401.4)$ (rounded down to 401)

5. Model Creation & Evaluation Metrics:

Simple Linear Regression: We cannot apply as we have more than one input.

Multiple Linear Regression:

fit_intercept	positive	R2 score
True	False	0.7894
True	True	0.7894
False	false	-0.3563
False	true	-0.3563

This model is not the right choice due to low predication score when compare to other models.

SVM

Hyper Parameter	linear (r value)	rbf (r value)	poly (r value)	sigmoid (r value)	Precomputed (r value)
C=0.01	-0.0888	-0.0896	-0.0895	-0.0895	Cannot be used for this dataset as it is not a square matrix
C=0.1	-0.0809	-0.0890	-0.0883	-0.0882	
C=10	0.4624	-0.0322	0.0387	0.0393	
C=100	0.6288	0.3200	0.6179	0.5276	
C=500	0.7631	0.6642	0.8263	0.4446	
C=1000	0.7649	0.8102	0.8566	0.2874	
C=2000	0.7440	0.8547	0.8605	-0.5939	
C=10000	0.7414	0.8779	0.8591	-34.151	
C=100000	0.7414	0.8724	0.8577	-3465.9	

This SVM model achieved its highest score only at an extremely high regularization value C= 10000. The sigmoid model collapses at this high range of C. The model bends heavily to fit the training points perfectly, making it fragile and like to fail in real world.

Decision Tree:

S.No.	Criterion	Max Features	Splitter	R Value
1	squared_error	None	best	0.6925
2	squared_error	None	Random	0.7418
3	squared_error	Sqrt	Best	0.6127

4	<i>squared_error</i>	Sqrt	Random	0.6890
5	<i>squared_error</i>	Log2	Best	0.7186
6	<i>squared_error</i>	Log2	Random	0.5637
7	<i>absolute_error</i>	None	best	0.6906
8	<i>absolute_error</i>	None	Random	0.7646
9	<i>absolute_error</i>	Sqrt	Best	0.7453
10	<i>absolute_error</i>	Sqrt	Random	0.7203
11	<i>absolute_error</i>	Log2	Best	0.7177
12	<i>absolute_error</i>	Log2	Random	0.6063
13	<i>friedman_mse</i>	None	best	0.6843
14	<i>friedman_mse</i>	None	Random	0.7221
15	<i>friedman_mse</i>	Sqrt	Best	0.7432
16	<i>friedman_mse</i>	Sqrt	Random	0.6982
17	<i>friedman_mse</i>	Log2	Best	0.6311
18	<i>friedman_mse</i>	Log2	Random	0.7322
19	<i>poisson</i>	None	best	0.6991
20	<i>poisson</i>	None	Random	0.7347
21	<i>poisson</i>	Sqrt	Best	0.6824
22	<i>poisson</i>	Sqrt	Random	0.6587
23	<i>poisson</i>	Log2	Best	0.6399
24	<i>poisson</i>	Log2	Random	0.6654

This model is not the right choice due to low predication score when compare to other models.

Random Forest:

critierion	n_estimators	max_depth	max_features	R2 score
<i>squared_error</i>	10	None	<i>sqrt</i>	0.8566
<i>squared_error</i>	50	None	log2	0.8692
<i>squared_error</i>	100	None	None	0.8546
<i>squared_error</i>	10	5	<i>sqrt</i>	0.8675
<i>squared_error</i>	50	50	log2	0.8686
<i>squared_error</i>	100	100	None	0.8658
<i>squared_error</i>	100	100	<i>sqrt</i>	0.8707
<i>squared_error</i>	50	50	log2	0.8720
<i>squared_error</i>	50	5	None	0.8827
<i>absolute_error</i>	10	None	<i>sqrt</i>	0.8578
<i>absolute_error</i>	50	None	log2	0.8693
<i>absolute_error</i>	100	None	None	0.8564
<i>absolute_error</i>	10	5	<i>sqrt</i>	0.8664
<i>absolute_error</i>	50	50	log2	0.8690
<i>absolute_error</i>	100	100	None	0.8527
<i>absolute_error</i>	100	100	<i>sqrt</i>	0.8688
<i>absolute_error</i>	50	50	log2	0.8710

absolute_error	10	5	None	0.8860
<i>friedman_mse</i>	10	None	<i>sqrt</i>	0.8561
<i>friedman_mse</i>	50	None	log2	0.8661
<i>friedman_mse</i>	100	None	None	0.8576
<i>friedman_mse</i>	10	5	<i>sqrt</i>	0.8624
<i>friedman_mse</i>	50	50	log2	0.8680
<i>friedman_mse</i>	100	100	None	0.8583
<i>friedman_mse</i>	100	100	<i>sqrt</i>	0.8670
<i>friedman_mse</i>	50	50	log2	0.8694
<i>friedman_mse</i>	10	5	None	0.8756
<i>Poisson</i>	10	None	<i>sqrt</i>	0.8637
<i>Poisson</i>	50	None	log2	0.8708
<i>Poisson</i>	100	None	None	0.8558
<i>Poisson</i>	10	5	<i>sqrt</i>	0.8706
<i>Poisson</i>	50	50	log2	0.8680
<i>Poisson</i>	100	100	None	0.8579
<i>Poisson</i>	100	100	<i>sqrt</i>	0.8678
<i>Poisson</i>	50	50	log2	0.8675
<i>Poisson</i>	10	5	None	0.8854

The Random Forest has consistently generated a similar score throughout the table. While achieving its peak performance with a max_depth=5, helping the model generalize well to new data and prevents overfitting compared to letting the trees grow indefinitely. This constraint ensures the model generalizes smoothly, handles the non-linear relationship of the dataset.

The best model for this dataset in the real time usage is **Random Forest**

6. Saving the best model

The model is deployed here

<https://github.com/sunandaadityaseshadri-bliip/6.MachineLearningRegressionAssignments/>